

NPTEL

Applied Accelerated Artificial Intelligence

Instructor: Dr. Satyajit Das
Dept. of Computer Science and Engineering
Indian Institute of Technology Guwahati, Assam, India

Segment 3: GPU Fundamentals & Modern AI Accelerators

Inside the engines of modern AI: from thousands of CUDA cores to Tensor Cores and beyond

Learning Objectives - Segment 3

01 Describe the SIMT (Single Instruction Multiple Threads) execution model of GPUs

02 Explain the CUDA thread hierarchy: threads, warps, blocks, and grids

03 Identify the role of Tensor Cores in matrix multiplication acceleration

04 Compare NVIDIA H100, AMD MI300X, Google TPU v4, and Graphcore IPU architectures

05 Select appropriate accelerators for training vs. inference workloads based on specifications

GPU Architecture: SIMT and the Streaming Multiprocessor

SIMT: Single Instruction, Multiple Threads

- GPU executes thousands of threads simultaneously in lockstep groups (warps)
- Warp = 32 threads executing the same instruction on different data (SIMD at scale)
- Warp divergence: if-else branches cause serialisation - avoid in GPU kernels

Streaming Multiprocessor (SM) - H100 SXM

- H100 SXM: 132 SMs, each with 128 CUDA cores (FP32) = 16,896 total CUDA cores
- Each SM: 4 Tensor Core units, 256 KB register file, 228 KB shared memory/L1
- SM executes 4 warp instructions simultaneously via 4 warp schedulers

CUDA Thread Hierarchy

- Thread < Warp (32) < Block (up to 1024) < Grid (millions)
- Shared memory is per-block: fast (no cache miss), programmer-managed
- Global memory (HBM): accessible by all threads, high latency, high bandwidth

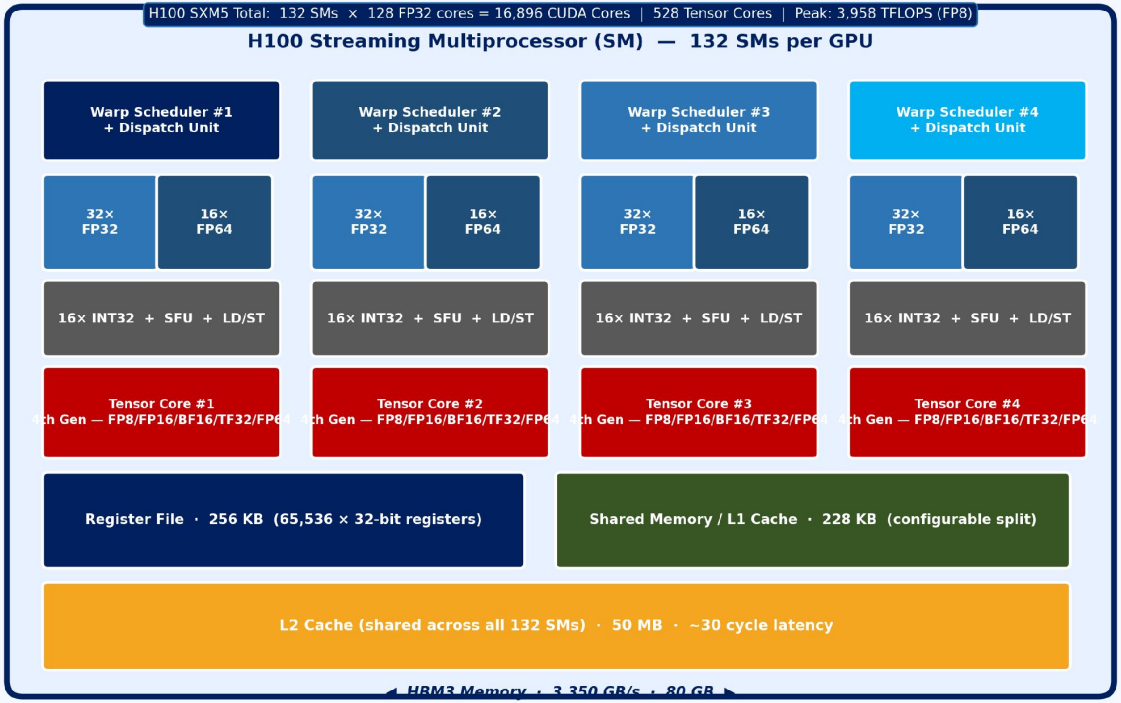


Fig 7: H100 Streaming Multiprocessor Architecture

Ref: NVIDIA H100 GPU Architecture Whitepaper (2022); Kirk & Hwu, "Programming Massively Parallel Processors," 4th Ed.

Tensor Cores: Accelerating the GEMM at the Heart of AI

What is a Tensor Core?

- Specialised fixed-function unit performing D = A*B + C (matrix multiply-accumulate) per cycle
- 4th Gen Tensor Core (H100): 1 TC performs 256 FP16 MACs per clock cycle
- H100 SXM5: 528 Tensor Cores x 256 ops/cycle x 1.98 GHz = ~270 TFLOPS (FP16)

Supported Precisions and Use Cases

- FP64: 67 TFLOPS - scientific simulation (double precision), not used in DL training
- TF32 (19-bit): 989 TFLOPS - drop-in FP32 replacement for training, no code change
- BF16/FP16: 1979 TFLOPS - standard for mixed-precision training (with FP32 master weights)
- FP8 (E4M3/E5M2): 3958 TFLOPS - emerging standard for LLM training (Transformer Engine)
- INT8/INT4: for quantised inference; INT4 = 7.9 PFLOPS - extreme throughput

Mixed Precision Training (AMP)

- Weights stored in FP32 (master copy); forward/backward pass in FP16 via Tensor Cores
- Loss scaling prevents FP16 underflow (gradients too small for 16-bit representation)
- torch.cuda.amp.autocast() enables automatic mixed precision with one line of code

Key: Tensor Cores are only effective for large matrix shapes - small GEMM sizes underutilise them

Ref: NVIDIA Hopper Architecture In-Depth (2022); Micikevicius et al., Mixed Precision Training, ICLR 2018

Tensor Cores: Accelerating the GEMM at the Heart of AI

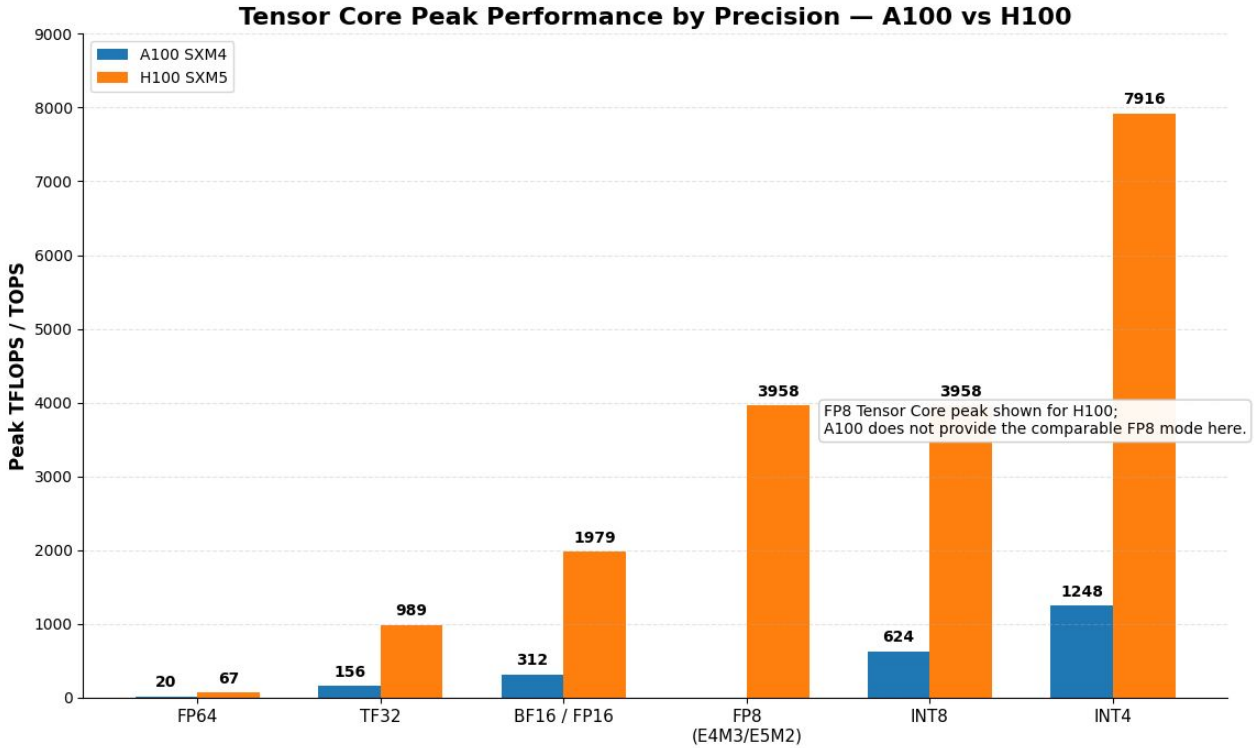


Fig 8: Tensor Core Precision Modes & TFLOPS

Ref: NVIDIA Hopper Architecture In-Depth (2022); Micikevicius et al., Mixed Precision Training, ICLR 2018

Beyond NVIDIA: AMD, TPU, Graphcore and Custom Accelerators

AMD MI300X (2023)

- 13 chiplets (3nm CDNA3 + 6x HBM3): 1.307 PFLOPS FP16
- 192 GB HBM3 memory @ 5.3 TB/s - largest GPU VRAM available
- ROCm software stack: PyTorch/JAX support via HIP (CUDA portability layer)
- Best for LLM inference (huge model fits in single GPU VRAM)

Google TPU v4 (2021)

- Custom ASIC: 275 TFLOPS (BF16), 32 GB HBM2e @ 1.2 TB/s
- TPU Pod v4: 4096 chips interconnected via custom 3D torus network
- Optimal for JAX/XLA workloads (TensorFlow, Flax), not general-purpose
- Systolic array architecture: data flows through mesh of MACs without control overhead

Graphcore IPU (Intelligence Processing Unit)

- 1472 independent processor tiles with local SRAM (for graph neural networks)
- BSP (Bulk Synchronous Parallel): compute phase alternates with communicate phase
- Ideal for sparse, irregular workloads (GNNs, symbolic AI) - poor for dense GEMM

AWS Trainium & Inferentia (Custom ASICs)

- Trainium2: 190.7 TFLOPS (FP16), optimised for AWS SageMaker training jobs
- Inferentia2: 190 TOPS (INT8), purpose-built for low-latency inference serving

Accelerator Selection Guide

- Large model training: H100 SXM or MI300X (highest HBM BW)
- Cost-efficient training: A100 80GB or H100 PCIe clusters
- Low-latency inference: Inferentia2, L40S, or quantised RTX 4090
- Batch inference: A10G, L4, T4 (Google Cloud) - best performance/dollar

Ref: AMD MI300X Architecture Whitepaper (2023); Google TPU v4 Jouppi et al., ISCA 2023

GPU Memory Management and VRAM Optimisation

VRAM is the Binding Constraint in LLM Training

- GPT-3 (175B params): FP16 weights = 350 GB VRAM; single H100 = 80 GB - needs 5+ GPUs
- VRAM usage = weights + gradients + optimiser states + activations (all must fit simultaneously)
- Adam optimiser: stores 3 copies per weight (param + momentum + variance) = 3x weight size

ZeRO (Zero Redundancy Optimiser) - Microsoft DeepSpeed

- ZeRO-1: shard optimiser states across GPUs - 4x memory reduction
- ZeRO-2: shard optimiser states + gradients - 8x memory reduction
- ZeRO-3: shard params + gradients + optimiser states - linear scaling with GPU count
- ZeRO-Offload: spill optimiser states to CPU RAM - enables 10x larger models on same GPU

Practical VRAM Budgeting Formula

- Training memory (bytes) = 16 x model_params (for FP16 + AMP Adam)
- Inference memory (bytes) = 2 x model_params (FP16) or 1 x model_params (INT8)
- Activation memory per layer proportional to batch_size x seq_len x hidden_dim

Tool: nvidia-smi --query-gpu=memory.used,memory.free --format=csv

Tool: torch.cuda.memory_summary() for detailed per-tensor breakdown

Ref: Rajbhandari et al., "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," SC'20

Segment 3 Summary: GPU Fundamentals & AI Accelerators

- ✓ GPUs execute thousands of threads in warps (32) via SIMT; occupancy and warp scheduling hide memory latency
- ✓ Tensor Cores perform matrix MACs per cycle: H100 reaches 3958 TFLOPS (FP8) - 600x vs single CPU core
- ✓ HBM3 (3.35 TB/s) is the defining bandwidth advantage of the H100; AMD MI300X offers 192 GB / 5.3 TB/s
- ✓ TPUs (systolic arrays), IPUs (tile-based), and cloud ASICs offer architecture diversity for specific workloads
- ✓ ZeRO-3 sharding and gradient checkpointing are essential techniques for training beyond single-GPU VRAM limits

Thank you!